

Q1: Which algorithm gave better average waiting time?

From our comparison logic:

```
Better Waiting Time: ${rr.avgWT < pr.avgWT ? "RR" :  
"Priority"}
```

So the answer depends on the dataset you ran.

But based on how our algorithms are written:

- **Round Robin** → usually *higher waiting time* because processes keep getting paused and re-queued
- **Priority Scheduling (preemptive in our code)** → usually *lower waiting time*, especially for high-priority processes

So in most cases in our project:

Priority Scheduling gave better average waiting time

Q2: Which algorithm gave better response time?

Same logic in our code:

```
Better Response Time: ${rr.avgRT < pr.avgRT ? "RR" :  
"Priority"}
```

But behavior-wise:

- **Round Robin** → very good response time (every process gets CPU quickly)
- **Priority** → low-priority processes may wait a lot before even starting

So generally:

Round Robin gave better response time

Q3: Did higher-priority processes gain significant advantage?

Yes, clearly — and our code enforces that hard:

```
return a.priority - b.priority;
```

- Smaller number = higher priority
- They always get picked first (and our version is actually **preemptive**, even though labeled non-preemptive)

So:

Yes, high-priority processes got a strong advantage (faster execution, lower waiting & response time)

Q4: Did Round Robin appear more balanced across all processes?

Yes, and our own analysis literally says:

“Round Robin: Highly fair, gives equal time slices”

Because:

- Every process gets CPU in cycles
- No one is skipped or delayed too much

So:

Yes, Round Robin is clearly more balanced and fair

Q5: Was starvation observed or likely in Priority Scheduling?

Our code checks this using:

```
if (prMaxWT > pr.avgWT * 1.5)
```

And conceptually:

- High-priority processes keep interrupting
- Low-priority ones may wait a *long* time

So:

Yes, starvation is likely in Priority Scheduling (especially for low-priority processes)

Q6: Which algorithm would you recommend for the tested workload, and why?

This depends on what your workload cares about:

- If your system needs:
 - fairness
 - no starvation
 - good response time**Round Robin is better**
- If your system needs:
 - fast handling of important tasks
 - priority-based execution**Priority Scheduling is better**

For our project specifically (since we're comparing fairness vs urgency):

Recommended answer:

- If your results showed better averages → choose that algorithm
- Otherwise:

Round Robin is the safer overall choice

because:

- no starvation
- fair CPU distribution
- consistent performance across all processes